

Table of Contents

Introduction

Design

Development

- Maze
- Weapons
- Pickups
- Enemies

Testing

Code

Disks/Instructions

Sources

Introduction

Initially, we were each working on our own individual senior projects. Once we realized that Unreal Engine 4 became free to use, we both became interested in using this game engine. We began learning how to use Unreal Engine 4 and began work on the game during the Summer of 2015. We officially switched our senior project to this collaborative effort in Unreal Engine shortly following summer break.

Design

Our game is best described as a Rogue-Light Third-Person Shooter. Rogue-Light as a genre refers to games with non-continuous gameplay sessions “runs” that are based on a combination of skilled play from the players while adapting to randomized challenges and boons. Third-Person Shooter refers to gameplay that involves an over-the-shoulder camera view on a player character that is equipped with ranged or melee attacks aimed by the player’s cursor.

This design concept lead our team towards two overarching design concepts that are the core mechanics of our game. Mechanic one is a randomly generated game world in which each run takes place. This provides a unique challenge for the player each time they decide to play it. The second is a power-up and weapon system in which all randomly rewarded boosts can be applied to the player character and will modify them according to the power-up’s type no matter what weapon they have equipped. These two features are what we consider to be the core of our game, and in order to call our project complete we knew we needed systems in place for both of them. Even if either system is not as feature complete as a release-level game there should be

demonstrable examples of the principals stated above in a system that is expandable for future development.

The map development was heavily discussed in the early stages of planning. We decided that the “maze”, as we came to call it, should consist of “rooms” placed randomly in the world and then interconnected by a network of “paths”. These rooms would not be individually randomly generated, but instead pre-built by level designers to create a pool of well-designed play environments that the world generator can pull from. The paths would then be randomly generated to connect these rooms together. At either end of the maze there would be a boss room and a spawn room, representing where the player starts and where the run ends. These cramped walkways connecting larger spaces are not unlike other dungeon crawler themed games, where the rooms and paths are cramped interior spaces; we wanted to avoid such a claustrophobic environment while preserving the layout and so decided on a set of canyons as our setting, which preserved the walls necessary for the layout, but allowed a wide open vertical environment.

The second mechanic is a combination of two fairly basic gaming concepts: equipable weapons and pickupable power-ups. The big idea behind what makes our game unique is that power-ups should provide a permanent benefit that does not have to only apply to a specific weapon or your general character, but instead should be able to adapt to and modify any equipment that the player is using. This mechanic meant that our power-ups and weapons should consist of a single parent class each and that these two parent classes are fully interactive, with a host of values the power-ups can modify on the weapon. From there we can inherit from these two parent classes to create power-ups with unique effects (such as multiple changes at once) and weapons with unique mechanics (a rocket launcher versus an assault rifle). It is then the responsibility of the game designer to create child weapons in such a way that their overridden

components of the parent weapon still utilize the parent's values in an interesting way. We will not touch on it much in this document because it is less relevant to the programming aspect of the project but creating interesting weapons using existing values is the main design challenge in creating the mechanics of this game.

Development

Maze

Development on the maze class started before this was even our senior project. Over the summer we began working on the game and started on the Maze first. Originally the maze was stored simply in a 2-dimensional array of Booleans (array in this document will almost always refer to a TArray, an Unreal library vector implementation), in which each position represented whether there was a floor in that location on the world grid. As the class developed it became apparent that in order to allow hand crafted rooms to be placed in the world we needed a new storage container for the elements of the world map. This resulted in a new data structure that consists of two arrays: one which contained all the rooms in the map and one which contained all the paths connecting them. A room knows its location and size, as well as its ID number, and also has a function Contains() which can be passed a location on the world grid and will return if that location is within its bounds. A path is further defined as an array of Walkways and an ID number. Walkways are a single location on the world grid that has a floor, store only their location, and have an overload for the == operator. This new data set allows many more useful features after the generation of the world map, such as the ability to loop through all the rooms in the game. World generation code was modified to develop both on the 2-dimensional array and the new data set, such that at the end of generation a copy of the map exists saved in both data structures. Nearly all the code that utilizes the 2d array data structure has been removed but it still exists in the parts of the generator that involve connecting the paths with rooms and removing dead ends. Completing the transition to the new data structure and removing the remaining 2d array code would be one of the first priorities of continued development to avoid saving the same data twice. Certain operations are slower in the new data structure, in which you

cannot directly get or set whether there is a floor on a certain part of the world grid, but as world generation happens at the beginning of the game and not during gameplay it is worth it to get more useful function out of the world map's data structure during runtime with the new structure.

The general process for generating the maze involves placing the starting and boss rooms, randomly attempting to place additional rooms around the empty space of the world grid, and then generating maze-like paths in the empty spaces left between the rooms. In order to ensure the rooms and paths fit together with exactly one space between them all corners of rooms and paths are placed on locations with coordinates that are [odd, odd]. After this initial process all [odd, odd] locations are guaranteed to be covered with a floor and two processes happen. First, all rooms are connected by a floor to each neighboring path. Second, all dead end paths are detected and removed. The end result is a map that consists of rooms interconnected by direct paths between them. There are occasionally multiple paths in and out of a room but all paths out of a room are guaranteed to lead into a different room. Instead of connecting each room to all adjacent paths, we considered connecting only enough to ensure there is a route to each room. This would be done by keeping a set of all elements (paths and rooms) that are already connecting and not creating new connections between elements already in the set.

The creation of the paths is done by one of two maze generators. The first implementation is a depth first implementation, where the tiles are placed in a snaking route and when it can no longer advance it backtracks to a location that it can. The second implementation is called PrimMaze and builds the maze using a breadth first algorithm. We made both algorithms so that we could choose which implementation produced a map that we liked more. In the end we decided to use the PrimMaze implementation because it produces less windy paths between the rooms. The different maze implementations are a good example of how modular and

expandable the code is for generating the maze. By changing dimensions, number of room attempts, and maze type we can generate vastly different maps using the same code.

Weapons

When coding the weapons we first began with a class called `WeaponBase`, which is consists of a tube that has a virtual `OnFire()` function to handle actually shooting mechanics. Normally the `WeaponBase` would be an abstract class, and `OnFire()` a pure virtual function, but Unreal doesn't support pure virtual functions because it instantiates one "default" object of each class. There is an Unreal Macro to make functions pure virtual but it is only for the programmer's benefit. The player character was then given a `WeaponBase` pointer, where it stores the currently equipped weapon. The `WeaponBase` also wants pointers to the character holding it, so that it can adjust to what the character is doing (such as tilting upwards when the character looks up); in order to provide these pointers we have an `InitializeConnection()` function that the character calls after getting a weapon, passing in its own information. Ideally this information would be passed through the weapon's constructor, but Unreal has some restrictions on constructor parameters that we did not want to deal with so instead we initialize the pointers to null and rely on the character to call this function. We have placed guards in our functions that use these pointers and they will not run if the weapon has not been initialized.

Weapons that inherit from the `WeaponBase` implement their own firing mechanic. Our two example weapons are a rocket launcher, whose `OnFire()` spawns a projectile, and an assault rifle, whose `OnFire()` does a line trace straight forward to check for objects in front of it. These firing behaviors use class variables from the `WeaponBase` to determine their properties. Our example properties are `Range`, `FireRate`, and `ProjectileSpeed`. One challenge we encountered is that not all properties have an obvious connection to all weapons, so if `ProjectileSpeed` is

modified while using an assault rifle, which spawns no projectiles, there is no effect. This is a design challenge, to incorporate values interestingly into different weapons and we could, for example, in future development implement a recoil mechanic that is unique to assault rifles whose values are calculated based on the ProjectileSpeed variable.

Expanding upon these systems is easily possible. We could add a shotgun to the code as easily as inheriting from the assault rifle class and having it trace multiple lines instead of just one, but at decreased fire rate and range. Values scaling differently for different weapons (like shotguns using range less effectively) are something we would like to implement differently. Currently we override the setter functions for these values to scale them when they are changed. For example, rocket launcher has an override function of SetRange() that divides the range by 3000. We would like to remove these overrides by modifying the value when it is used rather than when it is set. With less overrides we could divide the range before passing it to the projectile each time, it is (minutely) more computationally expensive but allows for more convenient inheritance. This would make the system more expandable, which a major goal of the project.

Pickups

The Pickup class was created for objects that the player could pick up within the game. Pickups included in the game at the moment include Powerups and Weapons. Each Pickup includes a collision component used to help determine whether the Player is in range to pick up the object, a mesh for the object, and an OnPickedUp function for handling interaction with the player character. The Consumable class is a child of the Pickup class and was intended to be used for any object that simply edited one of the character's numeric variables(for example increasing the character's health) and is then destroyed. In the end, the only thing Consumable really adds is an implementation of the OnPickedUp function that deals with destroying the

object. All of the actual functionality for picking up the object is dealt with in the `CanyonsCharacter` class. This is an element we would definitely change in the future; `CanyonsCharacter` is the class for the player character and should not handle pickup behavior. For interacting with pickups we added a `CollectionSphere` around the player, used to detect if any Pickups are in range of character. The function `CollectPickups` is called whenever the E key is pressed and deals with picking up any nearby Pickup. How the function works is any Actor that is within the `CanyonsCharacter`'s `CollectionSphere` is stored into an array. A for loop then casts each Actor to the `Consumable` class. If the cast is successful then the code appropriate for the specific Pickup is executed.

The visual components for the power-ups were added to objects through an Unreal Engine specific feature known as Blueprints. In this case, we simply created a Blueprint inherited from the `Consumable` class and set a material to be used for each `Consumable` object. This can also be done in code, but using Blueprints allows you to actually see how the object will look in game as its properties are being changed. Blueprints also support visual scripting, but that feature was not used for this portion of the project. We tried to stay away from using Blueprints because we preferred to write our code in C++ and thought it made for a better learning experience. C++ functions can be called and variables can be modified within Blueprints using Unreal's macros, which we used in a limited capacity in an attempt to comply with Unreal Engine 4's coding standards.

Enemies

One of the final classes we created was `Enemy`. The `Enemy` class is probably the most unfinished in our project. The only code in `Enemy` at the moment is an HP variable and code for attaching a weapon to an `Enemy` when it spawns. The idle animation, model, mesh, and material were all set

within the created Blueprint inherited from the Enemy class. One thing we would like to do later is make enemy inherit from CanyonsCharacter in some way since they share a lot of the same code and functionality. Not having an inheritance structure including CanyonsCharacter and Enemy resulted in some unneeded duplicate code and extra functions to distinguish between the two classes. The most important feature that our game is missing is working Enemy AI. We did not put priority on creating Enemy AI because the major goals of our project did not include doing anything interesting with Enemy AI. We also found, when researching the implementation of AI in Unreal Engine, that it relied heavily on Blueprints and Behavior models within the editor. We wanted to stick with as much C++ coding within Visual Studio as we could and were not prepared to commit the amount of time needed in order to learn how to appropriately implement Enemy AI in Unreal Engine.

Timeline	
Summer 2015	• Began Unreal Engine 4 research • Developed design concepts for the game • Researched random maze generation
Fall 2015	• Developed random maze generator • Implemented Tile spawning code for maze
Winter 2015/2016	• Began developing shooting mechanics and powerups
Spring 2016	• Finished shooting and powerups • Created enemy class • Randomly spawned enemies in maze

Testing

Testing was done periodically throughout the Development cycle. Most testing was done through playtesting the game within the unreal editor. We made sure to test each part of a function as we finished it. For example, in order to test Pickups we first made sure that pickups could be placed into the level. We then made sure that the player could walk next to any pickup and press e to destroy the pickup. A basic power-up that increases the HP value of the player was then created for testing. Finally, pickup functionality was implemented into weapons and power-ups that affect weapons were created and play tested. This helped us avoid building on top code that was not functioning properly and made the testing process go more smoothly.

Once the core of our game was completed we combined all the elements into a single playtesting experience. We did this by spawning an instance of each available weapon and power-up on a platform outside of where the maze spawns. We then play tested by running through the maze and killing enemies with each weapon and power-up combination.

// Fill out your copyright notice in the Description page of Project Settings.

#pragma once

```
#include "GameFramework/Actor.h"
#include "Tile.h"
#include "Maze.generated.h"
```

```
USTRUCT(BlueprintType)
struct CANYONS_API FMazeRow
{
    GENERATED_USTRUCT_BODY()

    //row[i].col[i] is true if place is filled, false if place is empty
    UPROPERTY(VisibleAnywhere, BlueprintReadWrite, Category = "Maze")
    TArray<bool> Col;

    FMazeRow(){}
};
```

```
USTRUCT(BlueprintType)
struct CANYONS_API FMazeArray
{
    GENERATED_USTRUCT_BODY()

    UPROPERTY(VisibleAnywhere, BlueprintReadWrite, Category = "Maze")
    TArray<FMazeRow> Row;

    FMazeArray(){}
};
```

```
USTRUCT()
struct CANYONS_API FRoom
{
    GENERATED_USTRUCT_BODY()

    uint16 OriginRow, OriginCol, High, Wide, SetID;
    //uint16 EntranceRow, EntranceCol; Rooms used to save their entrance tile, now it is part of the paths

    //Checks if the room contains a specified square
    bool Contains(uint16 Row, uint16 Col)
    {
        if (Row >= OriginRow && Row < OriginRow + High && Col >= OriginCol && Col < OriginCol + Wide)
        {
            return true;
        }
        return false;
    }

    //You can't have a modified constructor with USTRUCTS, so I'm just leaving the data entry to the Room creator, potential error point for sure
    FRoom(){}
};
```

```
USTRUCT()
struct CANYONS_API FWalkway
{
    GENERATED_USTRUCT_BODY()

    UPROPERTY(VisibleAnywhere, Category = "Maze")
    uint16 Row;
    UPROPERTY(VisibleAnywhere, Category = "Maze")
    uint16 Col;

    //== Operator, needed for TArray remove()
```

```

    bool operator==(const FWalkway &other) const
    {
        if (Row == other.Row && Col == other.Col)
        {
            return true;
        }
        return false;
    }

    //You can't have a modified constructor with USTRUCTS, so I'm just leaving the data entry to the Room creator, potential error point for sure
    FWalkway(){}
};

USTRUCT(BlueprintType)
struct CANYONS_API FPath
{
    GENERATED_USTRUCT_BODY()

    UPROPERTY(VisibleAnywhere, BlueprintReadWrite, Category = "Maze")
    TArray<FWalkway> Tiles;

    uint16 SetID;

    FPath(){}
};

UCLASS()
class CANYONS_API AMaze : public AActor
{
    GENERATED_UCLASS_BODY()

public:
    // Sets default values for this actor's properties
    AMaze();

    //Secondary constructor removed until needed again
    //AMaze(const FObjectInitializer &ObjectInitializer, uint16 High, uint16 Wide);

    //Initializes the data structures
    void Setup();

    // Called when the game starts or when spawned
    virtual void BeginPlay() override;

    // Called every frame
    virtual void Tick(float DeltaSeconds) override;

    //generation functions
    void GenerateMaze();

    void GenerateRooms(uint16 Tries); //Places rooms of random size in the maze, makes Tries attempts
    void PlaceRoom(uint16 OriginRow, uint16 OriginCol, uint16 High, uint16 Wide); //Places a room High tall and Wide wide with the upper left corner at (OriginRow, OriginCol), does nothing if room would connect with another open space

    void DepthFirstMaze(uint16 OriginRow, uint16 OriginCol); //Generates a maze using Depth-first algorithm starting at origin(X,Y)
    void DepthFirstMazeHelper(uint16 OriginRow, uint16 OriginCol, uint16 PathIndex);
    void PrimmMaze(uint16 OriginRow, uint16 OriginCol); //Generates a maze using Primm algorithm starting at origin(X,Y)

    void RemoveAllDeadEnds(); //Removes all dead ends from the maze component of the map
    void RemoveAllDeadEndsHelper(uint16 OriginRow, uint16 OriginCol); //Recursively removes a single dead end, calling itself on the dead end it creates upon removing one

```

```
void GenerateEntrances(); //Places paths for entering all rooms from nearby mazes. Will not generate
entrances for rooms that aren't adjacent to a path or room
uint16 FindTileOwner(uint16 OriginRow, uint16 OriginCol, bool &OutIsRoom); //Returns the SetID of the
set that contains the tile. Returns 0 if not found.
```

```
//Check if a position is in the maze and not yet filled in. Returns true only if both are true.
```

```
bool CheckAvailable(uint8 OriginRow, uint8 OriginCol)
{
    if (OriginRow > 0 && OriginRow < Height && OriginCol > 0 && OriginCol < Width)
    {
        if (MyMaze.Row[OriginRow].Col[OriginCol] == 0)
        {
            return true;
        }
    }
    return false;
}
```

protected:

```
//Function that spawns tiles to create the maze
UFUNCTION(BlueprintCallable, Category = "Maze")
void SpawnTiles();
//Function that spawns only the tiles that are Paths
UFUNCTION(BlueprintCallable, Category = "Maze")
void SpawnPaths();
//Function that spawns only the tiles that are Rooms and their entrances
UFUNCTION(BlueprintCallable, Category = "Maze")
void SpawnRooms();
```

```
void SpawnSingleTile(uint16 Row, uint16 Col);
```

```
//You should only change BossSize if you are re-making the room
void UpdateBossSize(uint8 NewSize);
```

```
//The Actor to be spawned by SpawnTiles
TSubclassOf<ATile> BlueprintVar;
```

```
UPROPERTY(VisibleAnywhere, BlueprintReadWrite, Category = "Maze")
FMazeArray MyMaze;
UPROPERTY(VisibleAnywhere, BlueprintReadWrite, Category = "Maze")
TArray<FRoom> Rooms;
UPROPERTY(VisibleAnywhere, BlueprintReadWrite, Category = "Maze")
TArray<FPath> Paths;
```

```
uint16 Height;
uint16 Width;
//The ID to be assigned to the next set created. Starts from 1. 0 represents no set containing a tile
uint16 NextID;
//Boss room size, defaults to 9x9, Boss room must be square
uint8 BossSize;
```

```
};
```

// Fill out your copyright notice in the Description page of Project Settings.

```
#include "Canyons.h"
#include "Maze.h"
#include <algorithm>
#include <time.h>
#include <array>

struct Entrance
{
    uint16 OriginRow, OriginCol, SetID1, SetID2, NewPath;

    bool operator==(const Entrance &other) const
    {
        if (SetID1 == other.SetID1 && SetID2 == other.SetID2)
        {
            return true;
        }
        else if (SetID1 == other.SetID2 && SetID2 == other.SetID1)
        {
            return true;
        }
        return false;
    }
};

// Sets default values
AMaze::AMaze(const FObjectInitializer &ObjectInitializer) : Super(ObjectInitializer), Height(31), Width(31)
{
    , BossSize(9), NextID(1)
    {
        static ConstructorHelpers::FObjectFinder<UBlueprint> PutNameHere(TEXT("/Game/Blueprints/TileBP.TileBP"));
        if (PutNameHere.Object) {
            BlueprintVar = (UClass*)PutNameHere.Object->GeneratedClass;
        }
        // Set this actor to call Tick() every frame. You can turn this off to improve performance if you don't need it.
        PrimaryActorTick.bCanEverTick = false;

        Setup();
    }

    //AMaze::AMaze(const FObjectInitializer &ObjectInitializer, uint16 High, uint16 Wide) : Super
    (ObjectInitializer), Height(High), Width(Wide), BossSize(9)
    //{
    //static ConstructorHelpers::FObjectFinder<UBlueprint> PutNameHere(TEXT("/Game/Blueprints/TileBP.TileBP"));
    //if (PutNameHere.Object) {
    //    BlueprintVar = (UClass*)PutNameHere.Object->GeneratedClass;
    //}
    //
    // // Set this actor to call Tick() every frame. You can turn this off to improve performance if you don't need it.
    // PrimaryActorTick.bCanEverTick = false;
    //
    // Height |= 1;
    // Width |= 1;
    //
    // Initialize();
    //}

    void AMaze::Setup()
    {
        srand(time(NULL));

        MyMaze.Row.SetNum(Height);
```



```

    for (uint16 i = 0; i < Height; ++i)
    {
        MyMaze.Row[i].Col.SetNumZeroed(Width);
    }

    PlaceRoom(1, 1, 9, 9);
    GenerateRooms(40);
    for (int Row = 1; Row < Height; Row += 2)
    {
        for (int Col = 1; Col < Width; Col += 2)
        {
            DepthFirstMaze(Row, Col);
        }
    }
    GenerateEntrances();
    RemoveAllDeadEnds();
    SpawnPaths();
    SpawnRooms();
}

// Called when the game starts or when spawned
void AMaze::BeginPlay()
{
    Super::BeginPlay();
}

// Called every frame
void AMaze::Tick(float DeltaTime)
{
    Super::Tick(DeltaTime);
}

void AMaze::GenerateRooms(uint16 Tries)//Places rooms of random size in the maze, makes Tries attempts
{
    uint16 Wide, High, OriginRow, OriginCol;
    uint16 MaxSize = BossSize - 3;
    for (uint16 i = 0; i < Tries; ++i){
        Wide = (rand() % MaxSize) + 2;
        High = (rand() % MaxSize) + 2;
        //Add 1 to random number if it is even. Implemented with bitwise OR to 1
        Wide |= 1;
        High |= 1;
        OriginRow = rand() % (Height - High - 1);
        OriginCol = rand() % (Width - Wide - 1);
        OriginRow |= 1;
        OriginCol |= 1;
        PlaceRoom(OriginRow, OriginCol, High, Wide);
    }
}

void AMaze::PlaceRoom(uint16 OriginRow, uint16 OriginCol, uint16 High, uint16 Wide)
{
    //Rooms must have odd dimensions and locations, otherwise return
    if (OriginRow % 2 == 0 || OriginCol % 2 == 0 || High % 2 == 0 || Wide % 2 == 0)
    {
        return;
    }
    //check if room space is already filled
    for (uint16 i = OriginRow; i < High + OriginRow; i = i + 2)
    {
        for (uint16 j = OriginCol; j < Wide + OriginCol; j = j + 2)
        {
            if (MyMaze.Row[i].Col[j] == true)

```

```

        {
            //If any of the spaces are filled, the room cannot be made and the function exits
            return;
        }
    }
}

for (uint16 i = OriginRow; i < High + OriginRow; ++i)
{
    for (uint16 j = OriginCol; j < Wide + OriginCol; ++j)
    {
        MyMaze.Row[i].Col[j] = true;
    }
}

FRoom Temp;
Temp.OriginCol = OriginCol;
Temp.OriginRow = OriginRow;
Temp.High = High;
Temp.Wide = Wide;
Temp.SetID = NextID;
++NextID;

Rooms.Add(Temp);
}

void AMaze::GenerateMaze()
{
}

//Generates a maze using Depth-first algorithm starting at origin(x,y)
void AMaze::DepthFirstMaze(uint16 OriginRow, uint16 OriginCol)
{
    //Maze must start at a place with an odd location
    if (OriginRow % 2 == 0 || OriginCol % 2 == 0)
    {
        return;
    }

    uint16 PathIndex = 0;

    if (MyMaze.Row[OriginRow].Col[OriginCol] == 0)
    {
        //create and record index of new path
        Paths.AddDefaulted(1);
        PathIndex = Paths.Num() - 1;
        Paths[PathIndex].SetID = NextID;
        ++NextID;

        MyMaze.Row[OriginRow].Col[OriginCol] = 1;
        FWalkway Temp;
        Temp.Col = OriginCol;
        Temp.Row = OriginRow;
        Paths[PathIndex].Tiles.Add(Temp);
    }
    else
    {
        return;
    }

    DepthFirstMazeHelper(OriginRow, OriginCol, PathIndex);
}

void AMaze::DepthFirstMazeHelper(uint16 OriginRow, uint16 OriginCol, uint16 PathIndex)
{
    MyMaze.Row[OriginRow].Col[OriginCol] = 1;

```

```

FWalkway Temp;
Temp.Col = OriginCol;
Temp.Row = OriginRow;
Paths[PathIndex].Tiles.Add(Temp);

uint8 Directions[4] = { 1, 2, 3, 4 };
std::random_shuffle(std::begin(Directions), std::end(Directions));
for (int8 i = 0; i < 4; i++)
{
    //TODO: IMPLEMENT USING NEW FUNCTION
    switch (Directions[i])
    {
    case 1: //Right
        if (OriginRow > 0 && OriginRow < Height && OriginCol > 0 && OriginCol + 2 < Width)
        {
            if (MyMaze.Row[OriginRow].Col[OriginCol + 2] == 0)
            {
                MyMaze.Row[OriginRow].Col[OriginCol + 1] = 1;
                FWalkway Temp;
                Temp.Col = OriginCol + 1;
                Temp.Row = OriginRow;
                Paths[PathIndex].Tiles.Add(Temp);

                DepthFirstMazeHelper(OriginRow, OriginCol + 2, PathIndex);
            }
        }
        break;
    case 2: //Down
        if (OriginRow > 0 && OriginRow + 2 < Height && OriginCol > 0 && OriginCol < Width)
        {
            if (MyMaze.Row[OriginRow + 2].Col[OriginCol] == 0)
            {
                MyMaze.Row[OriginRow + 1].Col[OriginCol] = 1;
                FWalkway Temp;
                Temp.Col = OriginCol;
                Temp.Row = OriginRow + 1;
                Paths[PathIndex].Tiles.Add(Temp);

                DepthFirstMazeHelper(OriginRow + 2, OriginCol, PathIndex);
            }
        }
        break;
    case 3: //Left
        if (OriginRow > 0 && OriginRow < Height && OriginCol - 2 > 0 && OriginCol < Width)
        {
            if (MyMaze.Row[OriginRow].Col[OriginCol - 2] == 0)
            {
                MyMaze.Row[OriginRow].Col[OriginCol - 1] = 1;
                FWalkway Temp;
                Temp.Col = OriginCol - 1;
                Temp.Row = OriginRow;
                Paths[PathIndex].Tiles.Add(Temp);

                DepthFirstMazeHelper(OriginRow, OriginCol - 2, PathIndex);
            }
        }
        break;
    case 4: //Up
        if (OriginRow - 2 > 0 && OriginRow < Height && OriginCol > 0 && OriginCol < Width)
        {
            if (MyMaze.Row[OriginRow - 2].Col[OriginCol] == 0)
            {
                MyMaze.Row[OriginRow - 1].Col[OriginCol] = 1;
                FWalkway Temp;
                Temp.Col = OriginCol;
                Temp.Row = OriginRow - 1;
            }
        }
    }
}

```

```

        Paths[PathIndex].Tiles.Add(Temp);

        DepthFirstMazeHelper(OriginRow - 2, OriginCol, PathIndex);
    }
    }
    break;
default:
    ;//does not execute, all values are 1-4
}
}
}

//Generates a maze using Primms algorithm starting at origin(x,y)
void AMaze::PrimMaze(uint16 OriginRow, uint16 OriginCol)
{
    //Maze must start at a place with an odd location
    if (OriginRow % 2 == 0 || OriginCol % 2 == 0)
    {
        return;
    }

    //Array of 4 values, two pair of coordinates, one representing the tile to be traveled to and one
    //representing the tile traveled across
    TArray<std::pair<std::pair<uint16, uint16>, std::pair<uint16, uint16>>> Coordinates;
    //number to keep track of the index of a new path, if created
    uint16 PathIndex = 0;

    if (MyMaze.Row[OriginRow].Col[OriginCol] == 0)
    {
        //create and record index of new path
        Paths.AddDefaulted(1);
        PathIndex = Paths.Num() - 1;
        Paths[PathIndex].SetID = NextID;
        ++NextID;

        //Add starting point to new path
        MyMaze.Row[OriginRow].Col[OriginCol] = 1;
        FWalkway Temp;
        Temp.Col = OriginCol;
        Temp.Row = OriginRow;
        Paths[PathIndex].Tiles.Add(Temp);
    }
    else
    {
        return;
    }

    //Add Up, Down, Left, Right to list
    if (CheckAvailable(OriginRow + 2, OriginCol))
    {
        Coordinates.Add(std::make_pair(std::make_pair(OriginRow + 2, OriginCol), std::make_pair(OriginRow + 1, OriginCol)));
    }
    if (CheckAvailable(OriginRow - 2, OriginCol))
    {
        Coordinates.Add(std::make_pair(std::make_pair(OriginRow - 2, OriginCol), std::make_pair(OriginRow - 1, OriginCol)));
    }
    if (CheckAvailable(OriginRow, OriginCol + 2))
    {
        Coordinates.Add(std::make_pair(std::make_pair(OriginRow, OriginCol + 2), std::make_pair(OriginRow, OriginCol + 1)));
    }
    if (CheckAvailable(OriginRow, OriginCol - 2))
    {
        Coordinates.Add(std::make_pair(std::make_pair(OriginRow, OriginCol - 2), std::make_pair(OriginRow, OriginCol - 1)));
    }
}

```

```

    OriginCol - 1)));
}

//Loop through list, randomly digging a tunnel, then adding its
while (Coordinates.Num() != 0)
{
    int Cur = rand() % Coordinates.Num();
    int OddRow = Coordinates[Cur].first.first;
    int OddCol = Coordinates[Cur].first.second;
    int WallRow = Coordinates[Cur].second.first;
    int WallCol = Coordinates[Cur].second.second;
    if (CheckAvailable(OddRow, OddCol))
    {
        MyMaze.Row[OddRow].Col[OddCol] = 1;
        FWalkway Temp;
        Temp.Col = OddCol;
        Temp.Row = OddRow;
        Paths[PathIndex].Tiles.Add(Temp);

        MyMaze.Row[WallRow].Col[WallCol] = 1;
        FWalkway Temp2;
        Temp2.Col = WallCol;
        Temp2.Row = WallRow;
        Paths[PathIndex].Tiles.Add(Temp2);

        if (CheckAvailable(OddRow + 2, OddCol))
        {
            Coordinates.Add(std::make_pair(std::make_pair(OddRow + 2, OddCol), std::make_pair(OddRow + 2, OddCol)));
        }
        if (CheckAvailable(OddRow - 2, OddCol))
        {
            Coordinates.Add(std::make_pair(std::make_pair(OddRow - 2, OddCol), std::make_pair(OddRow - 2, OddCol)));
        }
        if (CheckAvailable(OddRow, OddCol + 2))
        {
            Coordinates.Add(std::make_pair(std::make_pair(OddRow, OddCol + 2), std::make_pair(OddRow, OddCol + 1)));
        }
        if (CheckAvailable(OddRow, OddCol - 2))
        {
            Coordinates.Add(std::make_pair(std::make_pair(OddRow, OddCol - 2), std::make_pair(OddRow, OddCol - 1)));
        }
    }
    Coordinates.RemoveAt(Cur);
}

void AMaze::GenerateEntrances()
{
    TArray<Entrance> PotentialEntrances;
    //For each room add all spaces that connect that room to another tile to Potential Entrances
    for (int CurRoom = 0; CurRoom < Rooms.Num(); ++CurRoom)
    {
        //Check that the room is not on the border, preventing an entrance on that side
        if (Rooms[CurRoom].OriginCol > 1)
        {
            //Iterate along the side, along all odd spaces
            for (int i = 0; i < Rooms[CurRoom].High; i += 2)
            {
                //if the space across the way is filled, it is a valid entrance
                if (MyMaze.Row[Rooms[CurRoom].OriginRow + i].Col[Rooms[CurRoom].OriginCol - 2] == 1)
                {
                    bool bIsRoom1 = false, bIsRoom2 = false;

```

```

        Entrance Temp;
        //set the entrance's location
        Temp.OriginCol = Rooms[CurRoom].OriginCol - 1;
        Temp.OriginRow = Rooms[CurRoom].OriginRow + i;
        //decide which sets the entrances connect to, and if it connects two rooms together
        Temp.SetID1 = FindTileOwner(Rooms[CurRoom].OriginRow + i, Rooms[CurRoom].OriginCol,
bIsRoom1);
        Temp.SetID2 = FindTileOwner(Rooms[CurRoom].OriginRow + i, Rooms[CurRoom].OriginCol - 2,
bIsRoom2);

        //if you connect two rooms it will be a new path you are creating (represented by 0),
otherwise you save which path you are adding to
        if (bIsRoom1 == true && bIsRoom2 == true)
        {
            Temp.NewPath = 0;
        }
        else
        {
            if (bIsRoom1)
            {
                Temp.NewPath = Temp.SetID2;
            }
            else
            {
                Temp.NewPath = Temp.SetID1;
            }
        }

        PotentialEntrances.Add(Temp);
    }
}
if (Rooms[CurRoom].OriginRow > 1)
{
    for (int i = 0; i < Rooms[CurRoom].Wide; i += 2)
    {
        if (MyMaze.Row[Rooms[CurRoom].OriginRow - 2].Col[Rooms[CurRoom].OriginCol + i] == 1)
        {
            bool bIsRoom1 = false, bIsRoom2 = false;
            Entrance Temp;
            Temp.OriginCol = Rooms[CurRoom].OriginCol + i;
            Temp.OriginRow = Rooms[CurRoom].OriginRow - 1;
            Temp.SetID1 = FindTileOwner(Rooms[CurRoom].OriginRow, Rooms[CurRoom].OriginCol + i,
bIsRoom1);
            Temp.SetID2 = FindTileOwner(Rooms[CurRoom].OriginRow - 2, Rooms[CurRoom].OriginCol + i,
bIsRoom2);

            if (bIsRoom1 == true && bIsRoom2 == true)
            {
                Temp.NewPath = 0;
            }
            else
            {
                if (bIsRoom1)
                {
                    Temp.NewPath = Temp.SetID2;
                }
                else
                {
                    Temp.NewPath = Temp.SetID1;
                }
            }

            PotentialEntrances.Add(Temp);
        }
    }
}

```

```

    }
    if (Rooms[CurRoom].OriginCol + Rooms[CurRoom].Wide < Width - 3)
    {
        for (int i = 0; i < Rooms[CurRoom].High; i += 2)
        {
            if (MyMaze.Row[Rooms[CurRoom].OriginRow + i].Col[Rooms[CurRoom].OriginCol + Rooms[CurRoom].
Wide + 1] == 1)
            {
                bool bIsRoom1 = false, bIsRoom2 = false;
                Entrance Temp;
                Temp.OriginCol = Rooms[CurRoom].OriginCol + Rooms[CurRoom].Wide;
                Temp.OriginRow = Rooms[CurRoom].OriginRow + i;
                Temp.SetID1 = FindTileOwner(Rooms[CurRoom].OriginRow + i, Rooms[CurRoom].OriginCol +
Rooms[CurRoom].Wide - 1, bIsRoom1);
                Temp.SetID2 = FindTileOwner(Rooms[CurRoom].OriginRow + i, Rooms[CurRoom].OriginCol +
Rooms[CurRoom].Wide + 1, bIsRoom2);

                if (bIsRoom1 == true && bIsRoom2 == true)
                {
                    Temp.NewPath = 0;
                }
                else
                {
                    if (bIsRoom1)
                    {
                        Temp.NewPath = Temp.SetID2;
                    }
                    else
                    {
                        Temp.NewPath = Temp.SetID1;
                    }
                }

                PotentialEntrances.Add(Temp);
            }
        }
    }
    if (Rooms[CurRoom].OriginRow + Rooms[CurRoom].High < Height - 3)
    {
        for (int i = 0; i < Rooms[CurRoom].Wide; i += 2)
        {
            if (MyMaze.Row[Rooms[CurRoom].OriginRow + Rooms[CurRoom].High + 1].Col[Rooms[CurRoom].
OriginCol + i] == 1)
            {
                bool bIsRoom1 = false, bIsRoom2 = false;
                Entrance Temp;
                Temp.OriginCol = Rooms[CurRoom].OriginCol + i;
                Temp.OriginRow = Rooms[CurRoom].OriginRow + Rooms[CurRoom].High;
                Temp.SetID1 = FindTileOwner(Rooms[CurRoom].OriginRow + Rooms[CurRoom].High + 1, Rooms
[CurRoom].OriginCol + i, bIsRoom1);
                Temp.SetID2 = FindTileOwner(Rooms[CurRoom].OriginRow + Rooms[CurRoom].High - 1, Rooms
[CurRoom].OriginCol + i, bIsRoom2);

                if (bIsRoom1 == true && bIsRoom2 == true)
                {
                    Temp.NewPath = 0;
                }
                else
                {
                    if (bIsRoom1)
                    {
                        Temp.NewPath = Temp.SetID2;
                    }
                    else
                    {
                        Temp.NewPath = Temp.SetID1;
                    }
                }
            }
        }
    }
}

```

```

        }
    }
    PotentialEntrances.Add(Temp);
}
}
}
}
while (PotentialEntrances.Num() > 0)
{
    uint16 i = rand() % PotentialEntrances.Num();
    MyMaze.Row[PotentialEntrances[i].OriginRow].Col[PotentialEntrances[i].OriginCol] = 1;

    uint16 PathIndex;
    if (PotentialEntrances[i].NewPath == 0)
    {
        //create and record index of new path
        Paths.AddDefaulted(1);
        PathIndex = Paths.Num() - 1;
        Paths[PathIndex].SetID = NextID;
        ++NextID;

        //Add point to new path
        FWalkway Temp;
        Temp.Col = PotentialEntrances[i].OriginCol;
        Temp.Row = PotentialEntrances[i].OriginRow;
        Paths[PathIndex].Tiles.Add(Temp);
    }
    else
    {
        //get the index of the path with ID NewPath
        for (PathIndex = 0; PathIndex < Paths.Num(); ++PathIndex)
        {
            if (Paths[PathIndex].SetID == PotentialEntrances[i].NewPath)
            {
                break;
            }
        }

        //Add point to path
        FWalkway Temp;
        Temp.Col = PotentialEntrances[i].OriginCol;
        Temp.Row = PotentialEntrances[i].OriginRow;

        Paths[PathIndex].Tiles.Add(Temp);
    }
    Entrance Temp = PotentialEntrances[i];
    PotentialEntrances.RemoveSwap(Temp);
}
}

uint16 AMaze::FindTileOwner(uint16 OriginRow, uint16 OriginCol, bool &OutIsRoom)
{
    for (int i = 0; i < Rooms.Num(); ++i)
    {
        if (Rooms[i].Contains(OriginRow, OriginCol))
        {
            OutIsRoom = true;
            return Rooms[i].SetID;
        }
    }
    for (int i = 0; i < Paths.Num(); ++i)
    {
        FWalkway Temp;
        Temp.Col = OriginCol;
    }
}

```



```

    Temp.Row = OriginRow;
    if (Paths[i].Tiles.Contains(Temp))
    {
        OutIsRoom = false;
        return Paths[i].SetID;
    }
}
return 0;
}

void AMaze::UpdateBossSize(uint8 NewSize)
{
    //Boss room must have an odd dimension
    if (NewSize % 2 == 0)
    {
        return;
    }
    BossSize = NewSize;
}

void AMaze::RemoveAllDeadEnds()
{
    for (uint16 Row = 1; Row < Height; Row += 2)
    {
        for (uint16 Col = 1; Col < Width; Col += 2)
        {
            RemoveAllDeadEndsHelper(Row, Col);
        }
    }
}

void AMaze::RemoveAllDeadEndsHelper(uint16 OriginRow, uint16 OriginCol)
{
    uint8 Sum = MyMaze.Row[OriginRow + 1].Col[OriginCol] + MyMaze.Row[OriginRow].Col[OriginCol + 1] +
    MyMaze.Row[OriginRow - 1].Col[OriginCol] + MyMaze.Row[OriginRow].Col[OriginCol - 1];
    //If it is a dead end
    if (Sum == 1 || Sum == 0)
    {
        //Remove current tile
        MyMaze.Row[OriginRow].Col[OriginCol] = 0;
        FWalkway Temp;
        Temp.Col = OriginCol;
        Temp.Row = OriginRow;
        for (uint16 i = 0; i < Paths.Num(); ++i)
        {
            Paths[i].Tiles.Remove(Temp);
            if (Paths[i].Tiles.Num() == 0)
            {
                Paths.RemoveAt(i);
            }
        }

        //Remove the tile from which we were a dead end
        if (MyMaze.Row[OriginRow + 1].Col[OriginCol] == 1)
        {
            MyMaze.Row[OriginRow + 1].Col[OriginCol] = 0;
            FWalkway Temp;
            Temp.Col = OriginCol;
            Temp.Row = OriginRow + 1;
            for (uint16 i = 0; i < Paths.Num(); ++i)
            {
                Paths[i].Tiles.Remove(Temp);
                if (Paths[i].Tiles.Num() == 0)
                {
                    Paths.RemoveAt(i);
                }
            }
        }
    }
}

```

```

    }
    RemoveAllDeadEndsHelper(OriginRow + 2, OriginCol);
}
else if (MyMaze.Row[OriginRow].Col[OriginCol + 1] == 1)
{
    MyMaze.Row[OriginRow].Col[OriginCol + 1] = 0;
    FWalkway Temp;
    Temp.Col = OriginCol + 1;
    Temp.Row = OriginRow;
    for (uint16 i = 0; i < Paths.Num(); ++i)
    {
        Paths[i].Tiles.Remove(Temp);
        if (Paths[i].Tiles.Num() == 0)
        {
            Paths.RemoveAt(i);
        }
    }
    RemoveAllDeadEndsHelper(OriginRow, OriginCol + 2);
}
else if (MyMaze.Row[OriginRow - 1].Col[OriginCol] == 1)
{
    MyMaze.Row[OriginRow - 1].Col[OriginCol] = 0;
    FWalkway Temp;
    Temp.Col = OriginCol;
    Temp.Row = OriginRow - 1;
    for (uint16 i = 0; i < Paths.Num(); ++i)
    {
        Paths[i].Tiles.Remove(Temp);
        if (Paths[i].Tiles.Num() == 0)
        {
            Paths.RemoveAt(i);
        }
    }
    RemoveAllDeadEndsHelper(OriginRow - 2, OriginCol);
}
else if (MyMaze.Row[OriginRow].Col[OriginCol - 1] == 1)
{
    MyMaze.Row[OriginRow].Col[OriginCol - 1] = 0;
    FWalkway Temp;
    Temp.Col = OriginCol - 1;
    Temp.Row = OriginRow;
    for (uint16 i = 0; i < Paths.Num(); ++i)
    {
        Paths[i].Tiles.Remove(Temp);
        if (Paths[i].Tiles.Num() == 0)
        {
            Paths.RemoveAt(i);
        }
    }
    RemoveAllDeadEndsHelper(OriginRow, OriginCol - 2);
}
}
}

void AMaze::SpawnTiles()
{
    for (uint16 r = 0; r < Height; r++)
    {
        for (uint16 c = 0; c < Width; c++)
        {
            if (MyMaze.Row[r].Col[c])
            {
                SpawnSingleTile(r, c);
            }
        }
    }
}

```

```

    }
}

void AMaze::SpawnPaths()
{
    for (uint16 CurPath = 0; CurPath < Paths.Num(); ++CurPath)
    {
        for (uint16 CurTile = 0; CurTile < Paths[CurPath].Tiles.Num(); ++CurTile)
        {
            SpawnSingleTile(Paths[CurPath].Tiles[CurTile].Row, Paths[CurPath].Tiles[CurTile].Col);
        }
    }
}

void AMaze::SpawnRooms()
{
    for (uint16 CurRoom = 0; CurRoom < Rooms.Num(); ++CurRoom)
    {
        //for (uint16 Row = 0; Row < Rooms[CurRoom].High; ++Row)
        //{
        //    for (uint16 Col = 0; Col < Rooms[CurRoom].Wide; ++Col)
        //    {
        //        SpawnSingleTile(Rooms[CurRoom].OriginRow + Row, Rooms[CurRoom].OriginCol + Col);
        //    }
        //}

        TSubclassOf<ATile> TileClass = NULL;

        //TODO: Construct Blueprint String name programatically for only one ObjectFinder call
        if (Rooms[CurRoom].High == 3)
        {
            if (Rooms[CurRoom].Wide == 3)
            {
                static ConstructorHelpers::FObjectFinder<UBlueprint> TileBlueprint(TEXT("/Game/Blueprints/ RoomBP_3x3.RoomBP_3x3"));
                if (TileBlueprint.Object) {
                    TileClass = (UClass*)TileBlueprint.Object->GeneratedClass;
                }
            }
            else if (Rooms[CurRoom].Wide == 5)
            {
                static ConstructorHelpers::FObjectFinder<UBlueprint> TileBlueprint(TEXT("/Game/Blueprints/ RoomBP_3x5.RoomBP_3x5"));
                if (TileBlueprint.Object) {
                    TileClass = (UClass*)TileBlueprint.Object->GeneratedClass;
                }
            }
            else if (Rooms[CurRoom].Wide == 7)
            {
                static ConstructorHelpers::FObjectFinder<UBlueprint> TileBlueprint(TEXT("/Game/Blueprints/ RoomBP_3x7.RoomBP_3x7"));
                if (TileBlueprint.Object) {
                    TileClass = (UClass*)TileBlueprint.Object->GeneratedClass;
                }
            }
            else if (Rooms[CurRoom].High == 5)
            {
                if (Rooms[CurRoom].Wide == 3)
                {
                    static ConstructorHelpers::FObjectFinder<UBlueprint> TileBlueprint(TEXT("/Game/Blueprints/ RoomBP_5x3.RoomBP_5x3"));
                    if (TileBlueprint.Object) {
                        TileClass = (UClass*)TileBlueprint.Object->GeneratedClass;
                    }
                }
            }
        }
    }
}

```

```

        else if (Rooms[CurRoom].Wide == 5)
        {
            static ConstructorHelpers::FObjectFinder<UBlueprint> TileBlueprint(TEXT("/Game/Blueprints/ ⚡
RoomBP_5x5.RoomBP_5x5"));
            if (TileBlueprint.Object) {
                TileClass = (UClass*)TileBlueprint.Object->GeneratedClass;
            }
        }
        else if (Rooms[CurRoom].Wide == 7)
        {
            static ConstructorHelpers::FObjectFinder<UBlueprint> TileBlueprint(TEXT("/Game/Blueprints/ ⚡
RoomBP_5x7.RoomBP_5x7"));
            if (TileBlueprint.Object) {
                TileClass = (UClass*)TileBlueprint.Object->GeneratedClass;
            }
        }
    }
    else if (Rooms[CurRoom].High == 7)
    {
        if (Rooms[CurRoom].Wide == 3)
        {
            static ConstructorHelpers::FObjectFinder<UBlueprint> TileBlueprint(TEXT("/Game/Blueprints/ ⚡
RoomBP_7x3.RoomBP_7x3"));
            if (TileBlueprint.Object) {
                TileClass = (UClass*)TileBlueprint.Object->GeneratedClass;
            }
        }
        else if (Rooms[CurRoom].Wide == 5)
        {
            static ConstructorHelpers::FObjectFinder<UBlueprint> TileBlueprint(TEXT("/Game/Blueprints/ ⚡
RoomBP_7x5.RoomBP_7x5"));
            if (TileBlueprint.Object) {
                TileClass = (UClass*)TileBlueprint.Object->GeneratedClass;
            }
        }
        else if (Rooms[CurRoom].Wide == 7)
        {
            static ConstructorHelpers::FObjectFinder<UBlueprint> TileBlueprint(TEXT("/Game/Blueprints/ ⚡
RoomBP_7x7.RoomBP_7x7"));
            if (TileBlueprint.Object) {
                TileClass = (UClass*)TileBlueprint.Object->GeneratedClass;
            }
        }
    }
    else if (Rooms[CurRoom].High == 9)
    {
        if (Rooms[CurRoom].Wide == 9)
        {
            static ConstructorHelpers::FObjectFinder<UBlueprint> TileBlueprint(TEXT("/Game/Blueprints/ ⚡
RoomBP_9x9.RoomBP_9x9"));
            if (TileBlueprint.Object) {
                TileClass = (UClass*)TileBlueprint.Object->GeneratedClass;
            }
        }
    }
}

float x, y, z;
z = 0;
x = (Rooms[CurRoom].OriginRow) * 500;
y = (Rooms[CurRoom].OriginCol) * 500;
FVector TileLoc = FVector(x, y, z);
FRotator Rotation = FRotator(0, 0, 0);

FActorSpawnParameters SpawnInfo;
SpawnInfo.bNoCollisionFail = true;
SpawnInfo.Owner = this;

```

```
        SpawnInfo.bDeferConstruction = false;

        UWorld* const World = GetWorld();
        if (World && TileClass)
        {
            ATile* const SpawnedTile = World->SpawnActor<ATile>(TileClass, TileLoc, Rotation, SpawnInfo);
        }
    }
}

void AMaze::SpawnSingleTile(uint16 Row, uint16 Col)
{
    //Location of tile to be placed
    FVector TileLoc;

    float x, y, z;
    z = 0;
    x = Row * 500;
    y = Col * 500;
    TileLoc = FVector(x, y, z);
    FRotator Rotation = FRotator(0, 0, 0);

    FActorSpawnParameters SpawnInfo;
    SpawnInfo.bNoCollisionFail = true;
    SpawnInfo.Owner = this;
    SpawnInfo.bDeferConstruction = false;

    UWorld* const World = GetWorld();
    if (World)
    {
        ATile* const SpawnedTile = World->SpawnActor<ATile>(BlueprintVar, TileLoc, Rotation, SpawnInfo);
    }

    //AMaze* NewTile = World->SpawnActor(TileBP, &TileLoc, &Rotation, SpawnInfo);
}
```

```
// Fill out your copyright notice in the Description page of Project Settings.

#pragma once

#include "GameFramework/Character.h"
#include "Enemy.generated.h"

UCLASS()
class CANYONS_API AEnemy : public ACharacter
{
    GENERATED_BODY()

public:
    // Sets default values for this character's properties
    AEnemy();

    // Called when the game starts or when spawned
    virtual void BeginPlay() override;

    // Called every frame
    virtual void Tick(float DeltaSeconds) override;

    // Called to bind functionality to input
    virtual void SetupPlayerInputComponent(class UInputComponent* InputComponent) override;

    // Called by damage sources
    virtual void DealDamage(float Amount);

protected:
    class AWeaponBase* Weapon;

    UPROPERTY(VisibleAnywhere, Category = EnemyStat)
    float HP;
};
```

// Fill out your copyright notice in the Description page of Project Settings.

```
#include "Canyons.h"
#include "Enemy.h"
#include "Weapons/RocketLauncher.h"
```

// Sets default values

```
AEnemy::AEnemy()
{
    // Set this character to call Tick() every frame. You can turn this off to improve performance if you
    // don't need it.
    PrimaryActorTick.bCanEverTick = false;

    HP = 100;
}
```

// Called when the game starts or when spawned

```
void AEnemy::BeginPlay()
{
    Super::BeginPlay();
    FVector Location;
    FRotator Direction;
    Location = GetMesh()->GetSocketLocation("ShoulderSocket");
    if (Controller)
    {
        Direction = Controller->GetControlRotation();
    }

    //Direction.Yaw += 90;
    UWorld* const World = GetWorld();
    if (World)
    {
        FActorSpawnParameters SpawnParams;
        SpawnParams.Owner = this;
        SpawnParams.Instigator = Instigator;
        Weapon = World->SpawnActor<ARocketLauncher>(Location, Direction, SpawnParams);
        if (Weapon){
            Weapon->AttachRootComponentTo(GetMesh(), "ShoulderSocket", EAttachLocation::KeepWorldPosition);
            Weapon->InitializeConnection(NULL, this);
            Weapon->SetActorRelativeRotation(FRotator(-90, 0, 0));
        }
    }
}
```

// Called every frame

```
void AEnemy::Tick(float DeltaTime)
{
    Super::Tick(DeltaTime);
}
```

// Called to bind functionality to input

```
void AEnemy::SetupPlayerInputComponent(class UInputComponent* InputComponent)
{
    Super::SetupPlayerInputComponent(InputComponent);
}
```

// Called by damage sources

```
void AEnemy::DealDamage(float Amount)
{
    HP -= Amount;
    if (HP <= 0)
    {
        if (Weapon)
```

```
    {  
        Weapon->Destroy();  
    }  
    Destroy();  
}  
}
```



```
// Copyright 1998-2015 Epic Games, Inc. All Rights Reserved.
```

```
#pragma once
```

```
#include "GameFramework/Character.h"
```

```
#include "CanyonsCharacter.generated.h"
```

```
UCLASS(config=Game)
```

```
class ACanyonsCharacter : public ACharacter
```

```
{
    GENERATED_BODY()

    /** Camera boom positioning the camera behind the character */
    UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = Camera, meta = (AllowPrivateAccess = "true"))
    class USpringArmComponent* CameraBoom;

    /** Follow camera */
    UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = Camera, meta = (AllowPrivateAccess = "true"))
    class UCameraComponent* FollowCamera;

public:
    ACanyonsCharacter();

    /** Base turn rate, in deg/sec. Other scaling may affect final turn rate. */
    UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category=Camera)
    float BaseTurnRate;

    /** Base look up/down rate, in deg/sec. Other scaling may affect final rate. */
    UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category=Camera)
    float BaseLookUpRate;

    //Sphere around character used to pick up items
    UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = Powerup)
    USphereComponent* CollectionSphere;

    //Health of character
    UPROPERTY(VisibleAnywhere, BlueprintReadWrite, Category = PlayerStat)
    float HP;

    //Multiplier for speed
    UPROPERTY(VisibleAnywhere, BlueprintReadWrite, Category = PlayerStat)
    float SpeedFactor;

    //Base speed of character; will never fall below base speed
    UPROPERTY(VisibleAnywhere, BlueprintReadWrite, Category = PlayerStat)
    float BaseSpeed;

    virtual void Tick(float DeltaSeconds) override;

    /** Returns CameraBoom subobject **/
    FORCEINLINE class USpringArmComponent* GetCameraBoom() const { return CameraBoom; }

    /** Returns FollowCamera subobject **/
    FORCEINLINE class UCameraComponent* GetFollowCamera() const { return FollowCamera; }

    //called every frame
    //virtual void Tick(float DeltaSeconds) override;

protected:
    //called when e is pressed to pick up item
    UFUNCTION(BlueprintCallable, category = Powerup)
    void CollectPickups();

    UFUNCTION(BlueprintImplementableEvent, category = Powerup)
    void PowerUp();

    class AWeaponBase* Weapon;
```

```
/** Called for forwards/backward input */
void MoveForward(float Value);

/** Called for side to side input */
void MoveRight(float Value);

/**
 * Called via input to turn at a given rate.
 * @param Rate This is a normalized rate, i.e. 1.0 means 100% of desired turn rate
 */
void TurnAtRate(float Rate);

/**
 * Called via input to turn look up/down at a given rate.
 * @param Rate This is a normalized rate, i.e. 1.0 means 100% of desired turn rate
 */
void LookUpAtRate(float Rate);

/** Handler for when a touch input begins. */
void TouchStarted(ETouchIndex::Type FingerIndex, FVector Location);

/** Handler for when a touch input stops. */
void TouchStopped(ETouchIndex::Type FingerIndex, FVector Location);

//Firing handlers
void OnFire();
void EndFire();

// APawn interface
virtual void SetupPlayerInputComponent(class UInputComponent* InputComponent) override;
// End of APawn interface

void EquipRocket();
void EquipRifle();
};
```

```
// Copyright 1998-2015 Epic Games, Inc. All Rights Reserved.
```

```
#include "Canyons.h"
#include "CanyonsCharacter.h"
#include "ProjectileRocket.h"
#include "Weapons/RocketLauncher.h"
#include "Weapons/AssaultRifle.h"
#include "Consumable.h"
```

```
////////////////////////////////////
// ACanyonsCharacter
```

```
ACanyonsCharacter::ACanyonsCharacter()
{
    //set default values
    HP = 100;
    SpeedFactor = 0.75;
    BaseSpeed = 10;

    //create sphere for collecting objects
    CollectionSphere = CreateDefaultSubobject<USphereComponent>("CollectionSphere");
    CollectionSphere->AttachTo(RootComponent);
    CollectionSphere->SetSphereRadius(200);

    // Set size for collision capsule
    GetCapsuleComponent()->InitCapsuleSize(42.f, 96.0f);

    // set our turn rates for input
    BaseTurnRate = 45.f;
    BaseLookUpRate = 45.f;

    // Don't rotate when the controller rotates. Let that just affect the camera.
    bUseControllerRotationPitch = false;
    bUseControllerRotationYaw = true;
    bUseControllerRotationRoll = false;

    // Configure character movement
    GetCharacterMovement()->bOrientRotationToMovement = false; // Character moves in the direction of input ↯
    ...
    //GetCharacterMovement()->RotationRate = FRotator(0.0f, 2000.0f, 0.0f); // ...at this rotation rate
    GetCharacterMovement()->JumpZVelocity = 600.f;
    GetCharacterMovement()->AirControl = 0.2f;

    // Create a camera boom (pulls in towards the player if there is a collision)
    CameraBoom = CreateDefaultSubobject<USpringArmComponent>(TEXT("CameraBoom"));
    CameraBoom->AttachTo(RootComponent);
    CameraBoom->TargetArmLength = 300.0f; // The camera follows at this distance behind the character
    CameraBoom->bUsePawnControlRotation = true; // Rotate the arm based on the controller

    // Create a follow camera
    FollowCamera = CreateDefaultSubobject<UCameraComponent>(TEXT("FollowCamera"));
    FollowCamera->AttachTo(CameraBoom, USpringArmComponent::SocketName); // Attach the camera to the end of ↯
    the boom and let the boom adjust to match the controller orientation
    FollowCamera->bUsePawnControlRotation = false; // Camera does not rotate relative to arm

    // Note: The skeletal mesh and anim blueprint references on the Mesh component (inherited from ↯
    Character)
    // are set in the derived blueprint asset named MyCharacter (to avoid direct content references in C++)
}

////////////////////////////////////
// Input

void ACanyonsCharacter::SetupPlayerInputComponent(class UInputComponent* InputComponent)
{
    // Set up gameplay key bindings
```

```

    check(InputComponent);
    InputComponent->BindAction("Jump", IE_Pressed, this, &ACharacter::Jump);
    InputComponent->BindAction("Jump", IE_Released, this, &ACharacter::StopJumping);

    InputComponent->BindAction("CollectPickups", IE_Pressed, this, &ACanyonsCharacter::CollectPickups);

    //Set up tick
    PrimaryActorTick.bCanEverTick = false;

    // Set up shooting key bindings
    InputComponent->BindAction("Fire", IE_Pressed, this, &ACanyonsCharacter::OnFire);
    InputComponent->BindAction("Fire", IE_Released, this, &ACanyonsCharacter::EndFire);

    InputComponent->BindAxis("MoveForward", this, &ACanyonsCharacter::MoveForward);
    InputComponent->BindAxis("MoveRight", this, &ACanyonsCharacter::MoveRight);

    // We have 2 versions of the rotation bindings to handle different kinds of devices differently
    // "turn" handles devices that provide an absolute delta, such as a mouse.
    // "turnrate" is for devices that we choose to treat as a rate of change, such as an analog joystick
    InputComponent->BindAxis("Turn", this, &APawn::AddControllerYawInput);
    InputComponent->BindAxis("TurnRate", this, &ACanyonsCharacter::TurnAtRate);
    InputComponent->BindAxis("LookUp", this, &APawn::AddControllerPitchInput);
    InputComponent->BindAxis("LookUpRate", this, &ACanyonsCharacter::LookUpAtRate);

    // handle touch devices
    InputComponent->BindTouch(IE_Pressed, this, &ACanyonsCharacter::TouchStarted);
    InputComponent->BindTouch(IE_Released, this, &ACanyonsCharacter::TouchStopped);
}

void ACanyonsCharacter::TouchStarted(ETouchIndex::Type FingerIndex, FVector Location)
{
    // jump, but only on the first touch
    if (FingerIndex == ETouchIndex::Touch1)
    {
        Jump();
    }
}

void ACanyonsCharacter::TouchStopped(ETouchIndex::Type FingerIndex, FVector Location)
{
    if (FingerIndex == ETouchIndex::Touch1)
    {
        StopJumping();
    }
}

void ACanyonsCharacter::TurnAtRate(float Rate)
{
    // calculate delta for this frame from the rate information
    AddControllerYawInput(Rate * BaseTurnRate * GetWorld()->GetDeltaSeconds());
}

void ACanyonsCharacter::LookUpAtRate(float Rate)
{
    // calculate delta for this frame from the rate information
    AddControllerPitchInput(Rate * BaseLookUpRate * GetWorld()->GetDeltaSeconds());
}

void ACanyonsCharacter::MoveForward(float Value)
{
    if ((Controller != NULL) && (Value != 0.0f))
    {
        // find out which way is forward
        const FRotator Rotation = Controller->GetControlRotation();
        const FRotator YawRotation(0, Rotation.Yaw, 0);
    }
}

```

```
        // get forward vector
        const FVector Direction = FRotationMatrix(YawRotation).GetUnitAxis(EAxis::X);
        AddMovementInput(Direction, Value);
    }
}

void ACanyonsCharacter::MoveRight(float Value)
{
    if ( (Controller != NULL) && (Value != 0.0f) )
    {
        // find out which way is right
        const FRotator Rotation = Controller->GetControlRotation();
        const FRotator YawRotation(0, Rotation.Yaw, 0);

        // get right vector
        const FVector Direction = FRotationMatrix(YawRotation).GetUnitAxis(EAxis::Y);
        // add movement in that direction
        AddMovementInput(Direction, Value);
    }
}

void ACanyonsCharacter::OnFire()
{
    if (Weapon)
    {
        Weapon->OnFire();
    }
}

void ACanyonsCharacter::EndFire()
{
    if (Weapon)
    {
        Weapon->EndFire();
    }
}

void ACanyonsCharacter::CollectPickups()
{
    //array for storing every actor overlapping collection sphere
    TArray<AActor*> CollectedActors;
    //store all actors overlapping collection sphere into array
    CollectionSphere->GetOverlappingActors(CollectedActors);

    for (int i = 0; i < CollectedActors.Num(); i++)
    {
        AConsumable* const ConsumableActor = Cast<AConsumable>(CollectedActors[i]);
        if (ConsumableActor && ConsumableActor->bIsActive == true)
        {
            switch (ConsumableActor->PowerupId)
            {
            {
            case 1:
                HP = HP + ConsumableActor->BonusValue;
                break;
            case 2:
                if (Weapon)
                {
                    Weapon->SetRange(ConsumableActor->BonusValue);
                }
                break;
            case 3:
                if (Weapon)
                {
                    Weapon->SetProjectileSpeed(ConsumableActor->BonusValue);
                }
            }
            }
        }
    }
}
```

```
        break;
    case 4:
        if (Weapon)
        {
            Weapon->SetFireRate(ConsumableActor->BonusValue);
        }
        break;
    case 5:
        if (Weapon)
        {
            Weapon->ScaleRange(ConsumableActor->BonusValue);
        }
        break;
    case 6:
        if (Weapon)
        {
            Weapon->ScaleProjectileSpeed(ConsumableActor->BonusValue);
        }
        break;
    case 7:
        if (Weapon)
        {
            Weapon->ScaleFireRate(ConsumableActor->BonusValue);
        }
        break;
    case 8:
        EquipRocket();
        break;
    case 9:
        EquipRifle();
        break;
    case 10:
        if (Weapon)
        {
            Weapon->ModRange(ConsumableActor->BonusValue);
        }
        break;
    case 11:
        if (Weapon)
        {
            Weapon->ModProjectileSpeed(ConsumableActor->BonusValue);
        }
        break;
    case 12:
        if (Weapon)
        {
            Weapon->ModFireRate(ConsumableActor->BonusValue);
        }
        break;
    default:
        break;
    }
    ConsumableActor->OnPickedUp();
    ConsumableActor->bIsActive = false;
}

}

void ACanyonsCharacter::Tick(float DeltaSeconds)
{
    Super::Tick(DeltaSeconds);
}

void ACanyonsCharacter::EquipRocket()
```

```
{
    FVector Location;
    FRotator Direction;
    Location = GetMesh()->GetSocketLocation("ShoulderSocket");
    Direction = Controller->GetControlRotation();
    Direction.Yaw += 90;
    UWorld* const World = GetWorld();
    if (World)
    {
        FActorSpawnParameters SpawnParams;
        SpawnParams.Owner = this;
        SpawnParams.Instigator = Instigator;
        if (Weapon)
        {
            Weapon->Destroy();
        }
        Weapon = World->SpawnActor<ARocketLauncher>(Location, Direction, SpawnParams);
        if (Weapon){
            Weapon->AttachRootComponentTo(GetMesh(), "ShoulderSocket", EAttachLocation::KeepWorldPosition);
            Weapon->InitializeConnection(Controller, this);
        }
    }
}

void ACanyonsCharacter::EquipRifle()
{
    FVector Location;
    FRotator Direction;
    Location = GetMesh()->GetSocketLocation("ShoulderSocket");
    Direction = Controller->GetControlRotation();
    Direction.Yaw += 90;
    UWorld* const World = GetWorld();
    if (World)
    {
        FActorSpawnParameters SpawnParams;
        SpawnParams.Owner = this;
        SpawnParams.Instigator = Instigator;
        if (Weapon)
        {
            Weapon->Destroy();
        }
        Weapon = World->SpawnActor<AAssaultRifle>(Location, Direction, SpawnParams);
        if (Weapon){
            Weapon->AttachRootComponentTo(GetMesh(), "ShoulderSocket", EAttachLocation::KeepWorldPosition);
            Weapon->InitializeConnection(Controller, this);
        }
    }
}
```

// Fill out your copyright notice in the Description page of Project Settings.

#pragma once

```
#include "GameFramework/Actor.h"
#include "Enemy.h"
#include "WeaponBase.generated.h"
```

class ACanyonsCharacter;

UCLASS()

class CANYONS_API AWeaponBase : public AActor

{
 GENERATED_BODY()

public:

// Sets default values for this actor's properties
 AWeaponBase();

//Called by the equipped Character upon creation
 void InitializeConnection(AController *InController, ACanyonsCharacter *InCharacter);

void InitializeConnection(AController *InController, AEnemy *Enemy);

// Called when the game starts or when spawned
 virtual void BeginPlay() override;

// Called every frame
 virtual void Tick(float DeltaSeconds) override;

//called when fire button pressed
 virtual void OnFire();

//called when fire button released
 virtual void EndFire();

//endplay, clear timer
 virtual void EndPlay(const EEndPlayReason::Type EndPlayReason) override;

//called to reposition weapon
 //void UpdateDirection(FRotator Direction);

virtual void ModRange(float NewValue);
 virtual void ModProjectileSpeed(float NewValue);
 virtual void ModFireRate(float NewValue);

virtual void SetRange(float NewValue);
 virtual void SetProjectileSpeed(float NewValue);
 virtual void SetFireRate(float NewValue);

virtual void ScaleRange(float NewValue);
 virtual void ScaleProjectileSpeed(float NewValue);
 virtual void ScaleFireRate(float NewValue);

protected:

//fire the weapon
 virtual void Fire();

// Weapon properties
 float Range;
 float ProjectileSpeed;
 float FireRate;

float MaxRange;
 float MaxProjectileSpeed;


```
float MinFireRate;

//Time handle for shooting loop
FTimerHandle FiringTimerHandle;

//Controller of equipped character
AController *Controller;

//Character Actor
ACanyonsCharacter *Character;

AEnemy *EnemyCharacter;
};
```

```
// Fill out your copyright notice in the Description page of Project Settings.
```

```
#include "Canyons.h"  
#include "WeaponBase.h"
```

```
// Sets default values
```

```
AWeaponBase::AWeaponBase()
```

```
{  
    // Set this actor to call Tick() every frame. You can turn this off to improve performance if you don't need it.  
    PrimaryActorTick.bCanEverTick = true;  
  
    UStaticMeshComponent* TrimVisual = CreateDefaultSubobject<UStaticMeshComponent>(TEXT("VisualRepresentation"));  
    static ConstructorHelpers::FObjectFinder<UStaticMesh> TrimVisualAsset(TEXT("/Game/StarterContent/Shapes/Shape_Trim.Shape_Trim"));  
    if (TrimVisualAsset.Succeeded())  
    {  
        TrimVisual->SetStaticMesh(TrimVisualAsset.Object);  
    }  
    RootComponent = TrimVisual;  
  
    Controller = NULL;  
    Character = NULL;  
  
    Range = 0;  
    FireRate = 0;  
    ProjectileSpeed = 0;  
  
    MaxRange = 0;  
    MinFireRate = 0;  
    MaxProjectileSpeed = 0;  
}
```

```
//gets pointers to equipped character's information, should be run on equip
```

```
void AWeaponBase::InitializeConnection(AController *InController, ACanyonsCharacter *MyCharacter)
```

```
{  
    Controller = InController;  
    Character = MyCharacter;  
    EnemyCharacter = NULL;  
}
```

```
void AWeaponBase::InitializeConnection(AController *InController, AEnemy *Enemy)
```

```
{  
    Controller = InController;  
    EnemyCharacter = Enemy;  
    Character = NULL;  
}
```

```
// Called when the game starts or when spawned
```

```
void AWeaponBase::BeginPlay()
```

```
{  
    Super::BeginPlay();  
}
```

```
// Called every frame
```

```
void AWeaponBase::Tick( float DeltaSeconds )
```

```
{  
    Super::Tick(DeltaSeconds);  
  
    if (Controller)  
    {  
        FRotator Direction = Controller->GetControlRotation();  
        float temp = Direction.Pitch;  
        Direction.Pitch = Direction.Roll;  
    }  
}
```

```
        Direction.Roll = temp;
        Direction.Yaw += 90;
        RootComponent->SetWorldRotation(Direction);
    }
}

void AWeaponBase::OnFire()
{
    GetWorldTimerManager().SetTimer(FiringTimerHandle, this, &AWeaponBase::Fire, FireRate, true, 0.0f);
}

void AWeaponBase::EndFire()
{
    GetWorldTimerManager().ClearTimer(FiringTimerHandle);
}

void AWeaponBase::Fire()
{
}

void AWeaponBase::EndPlay(const EEndPlayReason::Type EndPlayReason)
{
    GetWorldTimerManager().ClearTimer(FiringTimerHandle);
}

void AWeaponBase::ModRange(float NewValue)
{
    if (NewValue + MaxRange > MaxRange)
    {
        Range = MaxRange;
    }
    else
    {
        Range = NewValue + MaxRange;
    }
}

void AWeaponBase::ModProjectileSpeed(float NewValue)
{
    if (NewValue + MaxProjectileSpeed > MaxProjectileSpeed)
    {
        ProjectileSpeed = MaxProjectileSpeed;
    }
    else
    {
        ProjectileSpeed = NewValue + MaxProjectileSpeed;
    }
}

void AWeaponBase::ModFireRate(float NewValue)
{
    if (NewValue + MinFireRate < MinFireRate)
    {
        FireRate = MinFireRate;
    }
    else
    {
        FireRate = NewValue + MinFireRate;
    }
}

void AWeaponBase::SetRange(float NewValue)
{
    if (NewValue > MaxRange)
    {
    }
```

```
        Range = MaxRange;
    }
    else
    {
        Range = NewValue;
    }
}

void AWeaponBase::SetProjectileSpeed(float NewValue)
{
    if (NewValue > MaxProjectileSpeed)
    {
        ProjectileSpeed = MaxProjectileSpeed;
    }
    else
    {
        ProjectileSpeed = NewValue;
    }
}

void AWeaponBase::SetFireRate(float NewValue)
{
    if (NewValue < MinFireRate)
    {
        FireRate = MinFireRate;
    }
    else
    {
        FireRate = NewValue;
    }
}

void AWeaponBase::ScaleRange(float NewValue)
{
    if (Range * NewValue > MaxRange)
    {
        Range = MaxRange;
    }
    else
    {
        Range = Range * NewValue;
    }
}

void AWeaponBase::ScaleProjectileSpeed(float NewValue)
{
    if (ProjectileSpeed * NewValue > MaxProjectileSpeed)
    {
        ProjectileSpeed = MaxProjectileSpeed;
    }
    else
    {
        ProjectileSpeed = ProjectileSpeed * NewValue;
    }
}

void AWeaponBase::ScaleFireRate(float NewValue)
{
    if (FireRate * NewValue < MinFireRate)
    {
        FireRate = MinFireRate;
    }
    else
    {
        FireRate = FireRate * NewValue;
    }
}
```

}

// Fill out your copyright notice in the Description page of Project Settings.

#pragma once

#include "Weapons/WeaponBase.h"
#include "AssaultRifle.generated.h"

/**

*

*/

UCLASS()

class CANYONS_API AAssaultRifle : public AWeaponBase

{
 GENERATED_BODY()

public:

AAssaultRifle();

// Called every frame

virtual void Tick(float DeltaSeconds) override;

protected:

// Create hitscan

virtual void Fire() override;

};

// Fill out your copyright notice in the Description page of Project Settings.

```
#include "Canyons.h"
#include "AssaultRifle.h"
#include "Enemy.h"
```

```
AAssaultRifle::AAssaultRifle()
```

```
{
    Range = 9000.f;
    FireRate = 0.1f;

    MaxRange = 20000.f;
    MinFireRate = 0.05f;
}
```

//Create hitscan

```
void AAssaultRifle::Fire(){
    if (Controller && Character)
    {
        FVector Location = RootComponent->GetSocketLocation("Muzzle");
        FVector EndPoint = Location + (Controller->GetControlRotation().Vector() * Range);
        FCollisionQueryParams RV_TraceParams = FCollisionQueryParams(FName(TEXT("RV_Trace")), true, this);
        RV_TraceParams.bTraceComplex = true;
        RV_TraceParams.bTraceAsyncScene = true;
        RV_TraceParams.bReturnPhysicalMaterial = false;
        RV_TraceParams.AddIgnoredActor(this);
        RV_TraceParams.AddIgnoredActor((AActor*)Character);
        FHitResult Hit(ForceInit);
        UWorld* const World = GetWorld();
        if (World)
        {
            World->LineTraceSingleByChannel(Hit, Location, EndPoint, ECC_Pawn, RV_TraceParams);
        }
        FVector TraceEnd;
        if (Hit.GetActor())
        {
            TraceEnd = Hit.ImpactPoint;
            AEnemy* const HitActor = Cast<AEnemy>(Hit.GetActor());
            if (HitActor)
            {
                HitActor->DealDamage(5);
            }
        }
        else
        {
            TraceEnd = EndPoint;
        }
        DrawDebugLine(GetWorld(), Location, TraceEnd, FColor::Red, true, 0.1f, 10);
    }
}

// Called every frame
void AAssaultRifle::Tick(float DeltaSeconds)
{
    Super::Tick(DeltaSeconds);
}
```

// Fill out your copyright notice in the Description page of Project Settings.

#pragma once

#include "Weapons/WeaponBase.h"

#include "RocketLauncher.generated.h"

/**

*

*/

UCLASS()

class CANYONS_API ARocketLauncher : public AWeaponBase

{

GENERATED_BODY()

public:

ARocketLauncher();

virtual void SetRange(float NewValue) override;

virtual void ModRange(float NewValue) override;

protected:

// Launch Projectile

virtual void Fire() override;

};

// Fill out your copyright notice in the Description page of Project Settings.

```
#include "Canyons.h"
#include "RocketLauncher.h"
#include "ProjectileRocket.h"

ARocketLauncher::ARocketLauncher()
{
    FireRate = 0.5f;
    ProjectileSpeed = 3000.f;
    Range = 3.f;

    MaxRange = 5.f;
    MinFireRate = 0.2f;
    MaxProjectileSpeed = 10000.f;
}

// Called every frame
void ARocketLauncher::Fire()
{
    if (Controller)
    {
        FVector Location;
        Location = RootComponent->GetSocketLocation("Muzzle");
        FRotator Direction = Controller->GetControlRotation();
        UWorld* const World = GetWorld();
        if (World)
        {
            FActorSpawnParameters SpawnParams;
            SpawnParams.Owner = this;
            SpawnParams.Instigator = Instigator;
            AProjectileRocket* const Projectile = World->SpawnActor<AProjectileRocket>(Location, Direction,
SpawnParams);
            if (Projectile)
            {
                FVector const LaunchDir = Direction.Vector();
                Projectile->InitVelocity(LaunchDir, ProjectileSpeed, Range);
            }
        }
    }
}

void ARocketLauncher::SetRange(float NewValue)
{
    NewValue *= (0.0003);
    Super::SetRange(NewValue);
}

void ARocketLauncher::ModRange(float NewValue)
{
    NewValue *= (0.0003);
    Super::ModRange(NewValue);
}
```

// Fill out your copyright notice in the Description page of Project Settings.

#pragma once

#include "GameFramework/Actor.h"

#include "ProjectileRocket.generated.h"

UCLASS()

class CANYONS_API AProjectileRocket : public AActor

{

GENERATED_BODY()

UPROPERTY(VisibleDefaultsOnly, Category = Projectile)

USphereComponent* CollisionComp;

public:

// Sets default values for this actor's properties

//AProjectileRocket();

AProjectileRocket(const FObjectInitializer& ObjectInitializer);

// Called when the game starts or when spawned

virtual void BeginPlay() override;

// Called every frame

virtual void Tick(float DeltaSeconds) override;

void InitVelocity(const FVector& ShootDirection, float NewSpeed = 0, float Range = 2);

//Projectile movement

UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = Movement)

UProjectileMovementComponent* ProjectileMovement;

protected:

USphereComponent* ExplosionSphere;

//Time handle for expire time

FTimerHandle ExpirationTimerHandle;

//void AProjectileRocket::MyOnHit(const AActor* SelfActor, const AActor* OtherActor, FVector NormalImpulse, const FHitResult& Hit)

UFUNCTION()

virtual void MyOnHit();

//override AActor's OnConstruction, runs immediately after creation

virtual void OnConstruction(const FTransform & Transform) override;

};

// Fill out your copyright notice in the Description page of Project Settings.

```
#include "Canyons.h"
#include "ProjectileRocket.h"
#include "CanyonsCharacter.h"
#include "Enemy.h"

// Sets default values
AProjectileRocket::AProjectileRocket(const FObjectInitializer& ObjectInitializer)
{
    // Set this actor to call Tick() every frame. You can turn this off to improve performance if you don't need it.
    PrimaryActorTick.bCanEverTick = true;
    SetActorEnableCollision(true);

    UStaticMeshComponent* PillVisual = CreateDefaultSubobject<UStaticMeshComponent>(TEXT(
    "VisualRepresentation"));
    static ConstructorHelpers::FObjectFinder<UStaticMesh> PillVisualAsset(TEXT("/Game/StarterContent/Shapes
    /Shape_NarrowCapsule.Shape_NarrowCapsule"));
    if (PillVisualAsset.Succeeded())
    {
        PillVisual->SetStaticMesh(PillVisualAsset.Object);
    }
    RootComponent = PillVisual;

    ProjectileMovement = ObjectInitializer.CreateDefaultSubobject<UProjectileMovementComponent>(this, TEXT(
    "ProjectileComp"));
    ProjectileMovement->InitialSpeed = 3000.f;
    ProjectileMovement->MaxSpeed = 3000.f;
    ProjectileMovement->bRotationFollowsVelocity = false;
    ProjectileMovement->bShouldBounce = true;
    ProjectileMovement->Bounciness = 0.3f;
    ProjectileMovement->ProjectileGravityScale = 0;

    TScriptDelegate<FWeakObjectPtr> OnHitFunc;
    OnHitFunc.BindUFunction(this, "MyOnHit");
    OnActorHit.Add(OnHitFunc);

    ExplosionSphere = CreateDefaultSubobject<USphereComponent>("ExplosionSphere");
    ExplosionSphere->AttachTo(RootComponent);
    ExplosionSphere->SetSphereRadius(0);
}

void AProjectileRocket::OnConstruction(const FTransform & Transform){
    RootComponent->SetRelativeScale3D(FVector(0.5f));
    RootComponent->SetRelativeRotation(FRotator(-90, 0, 0));
}

// Called when the game starts or when spawned
void AProjectileRocket::BeginPlay()
{
    Super::BeginPlay();
}

// Called every frame
void AProjectileRocket::Tick( float DeltaTime )
{
    Super::Tick( DeltaTime );
}

void AProjectileRocket::InitVelocity(const FVector& ShootDirection, float NewSpeed, float Range)
{
    if (NewSpeed <= 0)
```

```

    {
        NewSpeed = ProjectileMovement->InitialSpeed;
    }
    if (ProjectileMovement)
    {
        ProjectileMovement->Velocity = ShootDirection * NewSpeed;
        FRotator RotDirection = ShootDirection.Rotation();
        RotDirection.Pitch -= 90;
        RootComponent->SetRelativeRotation(RotDirection);
    }
    GetWorldTimerManager().SetTimer(ExpirationTimerHandle, this, &AProjectileRocket::MyOnHit, Range, false) ✓
    ;
}

//void AProjectileRocket::MyOnHit(const AActor* SelfActor, const AActor* OtherActor, FVector NormalImpulse, ✓
const FHitResult& Hit)
//Triggered on contact with something
void AProjectileRocket::MyOnHit()
{
    GetWorldTimerManager().ClearTimer(ExpirationTimerHandle);

    ///Rocket AOE Collision/Damage detection
    ExplosionSphere->SetSphereRadius(500);
    TArray<AActor*> ActorsInExplosion;
    ExplosionSphere->GetOverlappingActors(ActorsInExplosion);
    for (int i = 0; i < ActorsInExplosion.Num(); i++)
    {
        AEnemy* EnemyChar = Cast<AEnemy>(ActorsInExplosion[i]);

        if (EnemyChar)
        {
            AEnemy* EnemyChar = Cast<AEnemy>(ActorsInExplosion[i]);
            if (EnemyChar)
            {
                EnemyChar->DealDamage(50);
            }
        }
    }
    Destroy();
}

```

// Fill out your copyright notice in the Description page of Project Settings.

#pragma once

#include "GameFramework/Actor.h"
#include "Pickup.generated.h"

UCLASS()

class CANYONS_API APickup : public AActor

{
 GENERATED_BODY()

public:

// Sets default values for this actor's properties
 APickup();

//notes:

//EditAnywhere: allows editing in editor and other places

//BlueprintReadWrite: allows blueprints to read and write this variable

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = Pickup)

bool bIsActive; //true if powerup is available for pickup

//notes:

//VisibleDefaultsOnly: component can be seen in editor and other places

//BlueprintReadOnly: only allows blueprints to read component

UPROPERTY(VisibleDefaultsOnly, BlueprintReadOnly, Category = Pickup)

TSubobjectPtr<USphereComponent> BaseCollisionComponent; //root component for powerup; used for collision

UPROPERTY(VisibleDefaultsOnly, BlueprintReadOnly, Category = Pickup)

TSubobjectPtr<UStaticMeshComponent> PickupMesh; //mesh for powerup

//notes:

//BlueprintNativeEvent: able to override in blueprint; use default function if no override given

UFUNCTION()

virtual void OnPickedUp();

// Called when the game starts or when spawned

virtual void BeginPlay() override;

// Called every frame

virtual void Tick(float DeltaSeconds) override;

};

// Fill out your copyright notice in the Description page of Project Settings.

```
#include "Canyons.h"
#include "Pickup.h"
```

// Sets default values

APickup::APickup()

```
{
    // Set this actor to call Tick() every frame. You can turn this off to improve performance if you don't
    // need it.
    PrimaryActorTick.bCanEverTick = true;

    //powerup is active when spawned
    bIsActive = true;

    //create sphere component to deal with powerup collision
    BaseCollisionComponent = CreateDefaultSubobject<USphereComponent>(TEXT("BasicSphereComponent"));

    //set sphere as the root component
    RootComponent = BaseCollisionComponent;

    //create static mesh component
    PickupMesh = CreateDefaultSubobject<UStaticMeshComponent>(TEXT("PickupMesh"));

    //attach StaticMeshComponent to RootComponent
    PickupMesh->SetCollisionEnabled(ECollisionEnabled::Type::NoCollision);
    PickupMesh->AttachTo(RootComponent);
}
```

// Called when the game starts or when spawned

void APickup::BeginPlay()

```
{
    Super::BeginPlay();
}
```

// Called every frame

void APickup::Tick(float DeltaTime)

```
{
    Super::Tick(DeltaTime);
}
```

////note: FunctionName_Implementation() b/c BlueprintNativeEvent

void APickup::OnPickedUp()

```
{
}
}
```

// Fill out your copyright notice in the Description page of Project Settings.

#pragma once

#include "Pickup.h"

#include "Consumable.generated.h"

/**

*

*/

UCLASS()

class CANYONS_API AConsumable : public APickup

{

GENERATED_BODY()

public:

AConsumable();

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = Powerup)
float BonusValue;

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = Powerup)
int32 PowerupId;

//override OnPickedUp function (implementation because blueprint native event)
virtual void OnPickedUp() override;

};

// Fill out your copyright notice in the Description page of Project Settings.

```
#include "Canyons.h"  
#include "Consumable.h"
```

```
AConsumable::AConsumable()  
{  
    BonusValue = 0;  
    PowerupId = 0;  
}
```

```
void AConsumable::OnPickedUp()  
{  
    Destroy();  
}
```


Disks/Instructions

Disks:

Game Disk - Executable Packaged Game

Source Disk - Source Code and Unreal Project File

Play Instructions:

1. Insert Game Disk
2. (Optional) For significantly faster load times, copy disk files onto hard drive.
3. Run "Canyons.exe"

Sources

Unreal Engine 4 Documentation: <https://docs.unrealengine.com/latest/INT/>

Unreal Engine 4 AnswerHub: <https://answers.unrealengine.com/>

Unreal Engine 4 Youtube: <https://www.youtube.com/channel/UCBobmJyzsJ6Ll7UbfhI4iwQ>